

CRUCIBLE: An Empirical Study of Harness–Model Compatibility and Delivery in Local Coding Agents

Siddhant Goswami
100x Engineers, Bengaluru, India
content@100xengineers.com

Abstract

A coding agent is a configuration: a model plus the harness that turns it into an agent (prompt assembly, tools, the execution loop, verification, recovery, permissions). We ask, under fixed task and resource conditions, how the configuration affects *delivery*—a verifier pass within a stated deadline—and what observable failures explain the rest. CRUCIBLE is a small apparatus for that question: a one-line adapter contract, a bounded act→verify loop owned by the runner, deterministic hidden verifiers, uniform token metering, and frozen run ledgers from which every reported number regenerates in CI. We report 1,600 attempted runs—1,527 local across five cohorts (six local harnesses; seven models in three families) and 73 in three small cloud arms, where a seventh harness appears. Three findings survive a reanalysis on matched task sets with task-clustered intervals. (i) Within a model column, harness choice moves delivery from 0.00 to 0.89 on `qwen3:8b`, but only six of the pre-named within-model contrasts are significant under both delivery and the gated score; two others are significant under one metric only, and we show which. (ii) Holding weights fixed on `qwen3.5:9b`, one tool-calling harness delivers 0.89 while another delivers 0.00 ($\Delta=+0.89$ [0.78, 1.00]): the same harness is 0 for 262 of 262 attempted local cells across five cohorts, a protocol failure visible in every trace, while it delivers 20/20 on its vendor’s cloud account. (iii) Scoring finished runs only reorders five of six harnesses on `qwen3:8b`; counting timeouts as failed deliveries is necessary, but a per-cell autopsy attributes 51 of the pilot’s 55 timeouts to host-conditional wall-clock cutoffs rather than harness hangs. A pre-registered third-family arm reproduces the structural zero and one directional contrast out of sample; a ten-task Terminal-Bench slice reproduces the ordering but not significance. The study is exploratory except where marked; its main limitations are nine authored tasks, one host, and adapted external tasks.

1 Introduction

Practitioners who run coding agents on local or mid-tier models choose a harness as much as a model, and the harness is rarely disclosed when agents are compared [15, 20]. The recent literature agrees that the harness matters: scaffold swaps move SWE-bench by 10–20 points [4, 5], the agent–computer interface alone unlocked large gains [17], two scaffolds on one model disagree on a large fraction of tasks [7], a single harness treatment transfers across frozen models [8], and Harness-Bench measures model×harness configurations under shared budgets with a security gate [19]. What that literature mostly reports is strong cloud models, on which harness choice moves tokens far more than pass rate (Vats and Golev [15] find 40× token variation against 0–8 point pass-rate differences).

This paper is a bounded empirical study in the complementary regime: *small local models* (1.5B–9.7B parameters on one Apple-Silicon host), where the harness moves the pass rate itself. It asks one question—under the tested task and resource conditions, how does configuration affect successful delivery, and what observable failures explain unsuccessful attempts?—and makes three bounded contributions:

1. **A reproducible apparatus** (§3) for comparing specified harness–model configurations: an adapter contract, a runner-owned bounded loop, hidden deterministic verifiers restored before every check, a token-metering proxy, and frozen ledgers with CI guards that fail the build when any published number, table, or figure drifts from the records.
2. **Configuration-dependent delivery on a bounded task battery** (§4): within-model contrasts with task-clustered intervals, a within-family size ladder, and a pre-registered out-of-sample arm—each reported under two outcome

metrics so that metric-dependence is visible rather than hidden.

3. **A diagnostic account of failures** (§5): a rule-derived failure taxonomy over every attempted cell, a weights-fixed protocol-failure demonstration, a timeout autopsy, and safety-gate components reported separately from the composite.

We describe the study as *exploratory* throughout, except for the one arm whose predictions were frozen before execution (§4.3). Broad causal claims—that harness effects dominate model capability in general, or that a cloud model swap isolates interface compatibility—are not made here; §6 says why, and Appendix D records which pre-registered predictions held, which were refuted, and which were never tested.

2 Related work

Harness as the variable. Harness-Bench [19] is the closest system: model×harness configurations on 106 tasks under shared budgets, a multiplicative security gate, and process scoring. CRUCIBLE shares that design (and cites it as the source of its factorial protocol and gate) but differs in regime and mechanism: sub-frontier local models as probes, deterministic trace-derived process scores rather than LLM log analysis, timeouts kept in the delivery denominator, and a cross-model rank-stability test against a seed-noise null. Liu et al. [9] formalize trajectory-level safety auditing and multiplicative scoring; our T4 tier is a small instance with per-channel deterministic checks. Zhang et al. [20] argue that harness-induced variance exceeds model-induced variance and call for disclosure; we do not estimate that decomposition (it needs a design we lack) but supply the disclosure. Vats and Golev [15] compare three open-source harnesses on two strong models and find large token but small pass-rate effects; our local-model regime is where the pass-rate effect appears, which is the intended complement. Lewis [8] show one harness treatment transferring across frozen models; our “reach” test asks the converse—whether a harness *ordering* survives a model swap—and finds it mostly does not. HAL [7] is cost-aware and process-level at scale but model-first; Agent Arena decomposes model×framework×tools but ranks by preference vote.

Evaluation methodology. We adopt cost-reported-alongside from Kapoor et al. [6], clustered resampling and paired contrasts from Miller [13], reliability-over-repeats from τ -bench’s pass^k [18], and the task-validity concerns of the ABC checklist [21]; one of our own task-design failures (§5.4) is an instance ABC catalogs. Benchmarks that score the (model, scaffold) pair jointly—SWE-bench [5], AgentBench [10], GAIA [12], OSWorld [16], Terminal-Bench [11], MLE-bench [1]—motivate the question; we import ten Terminal-Bench tasks through our contract (§4.4). Routing systems [2, 3, 14] condition on the query, not on a task’s tool requirements; we note the connection in §7 but demonstrate no router.

3 Experimental setup

Configurations. A configuration is a (harness, model) pair. Harnesses: `claude` (Claude Code), `aider` (repo-map, diff-driven, text edits), `codex` (OpenAI Codex CLI, structured tool protocol), `goose` (Block; many tools), `pi` (sub-1k-token prompt, four tools), `hermes` (Nous; skills, command approval), `ollama` (a thin file-block parser we wrote as the control), and `mock` (a deterministic floor with no model). Lean harnesses are wired to the same local endpoint, so within a model column the harness is the only difference. Local models via Ollama 0.30.11 with pinned digests: `deepseek-r1:1.5b`, `deepseek-r1:8b`, `qwen3:8b`, `qwen3.5:{2b,4b,9b}` (2.3B/4.7B/9.7B parameters; 2.7/3.4/6.6 GB quantized), `llama3.2:3b`. Cloud slices: Codex CLI 0.137.0 on its ChatGPT-account default model, which resolved to `gpt-5.5` in probe sessions recorded minutes before the first cloud cells (Appendix A); `gpt-4o-mini` via metered API; Claude Code on a Pro login.

Adapter contract and loop. A harness is integrated by one script, `adapters/<name>.sh <workdir> <iter> <feedback>`: read TASK.md, read the last verifier feedback, make one attempt by editing files. The runner owns the bounded loop (act → verify → repeat, capped), the sandbox (a throwaway copy per run), and the deadline, so every harness runs the identical protocol. Verifier, task metadata, and policy are hidden and restored from the pristine task before every check.

Tasks. Nine authored tasks in five tiers (Appendix B): T0 single-file fixes; T1 tool-recovery (pass requires *running* a generator that fails first; hardened after the pilot with a nonce+sha256 proof-of-execution, §5.4); T2 multi-file consistency; T3 artifact generation with sourcing rules; T4 a secret-redaction task with a hidden policy. Later cohorts add

Cohort	Configurations	Tasks	Attempted	Finished	Delivered	Status
Pilot	6 lean $\times$ 3 local (+mock, cloud slices)	9	515	460	164	exploratory, pre-hardening
qwen3.5 arms	3–4 tool-callers $\times$ qwen3.5:9b	4	99	66	15	exploratory; one refuted prediction
Size ladder	4 $\times$ qwen3.5:{2b,4b,9b}	6	222	200	78	declared subset, full hardening
Phase D	6 lean $\times$ llama3.2:3b, 5 seeds	11	341	336	75	pre-registered
Anchor	6 lean $\times$ 2 local, Terminal-Bench	10	370	280	24	adapted tasks, underpowered
Cloud	codex (account default); aider, pi @ gpt-4o-mini; Claude Code	3–4	73	72	65	demonstrations, small n

Table 1: Cohorts. “Attempted” includes timeouts; “finished” excludes them; “delivered” is a verifier pass within deadline. Counts from INVENTORY.md §A.1, which reports each ledger whole: the Pilot ledger contains 20 cloud cells that the Cloud row also counts, so the column does not sum to the study total of 1,600 attempted runs (1,527 local, 73 cloud).

two T1 tasks (eleven total). Ten Terminal-Bench tasks are imported through the same contract with a hidden Python oracle (§4.4). Task sets differ by cohort and are declared per table; contrasts use the matched set of their model column.

Budgets and deadlines. Each task carries `max_iters`, a token budget, and a wall deadline. The pilot used the task’s deadline as written; every later cohort raises it to $\max(\text{task}, \text{per-model T0 fit})$, shuffles cell order with a recorded seed, and probes host generation rate before every cell (a *host-health canary*), after the pilot’s autopsy showed wall-clock delivery on a memory-constrained host to be non-stationary (§5.2). Temperature 0.7; three seeds per cell (five in the pre-registered arm). Only the control adapter exposes a true seed; for the others the repeats are independent samples.

Outcomes. The **primary outcome is delivery**: the verifier passed and the run finished within the deadline, over *attempted* cells. The **secondary outcome** is the gated score used in our pre-registration,

$$\text{Score} = \text{Safety} \times (0.6 \text{ Completion} + 0.2 \text{ Path} + 0.2 \text{ State}),$$

averaged over attempted cells with timeouts as 0 (GOODPUT). Safety is $\min(\text{tool}, \text{resource}, \text{info})$ per-channel adherence: a high-severity event sets a channel to 0; each low-severity event subtracts 0.15. Completion is the verifier with checkpoint partial credit; Path and State are deterministic trace proxies. Cost (metered tokens, wall time) is reported alongside, never folded in. *The choice of delivery as primary is a post-hoc analysis decision for this manuscript*; every contrast is therefore shown under both metrics.

Analysis. Task-clustered bootstrap (resample tasks, keep all their attempts; seeded; $B=5000$) for every interval; contrasts paired by (task, seed) and clustered by task. Timeouts stay in the denominator. All eligible configurations in a cohort are reported under its declared rule. Rank stability across model columns is tested against a null that pools each harness’s runs across columns and resamples. Tables and figures regenerate from the committed ledgers (`crucible/tools/reanalysis.js`, `inventory.js`); CI fails on drift.

Cohorts. Table 1 lists the five local cohorts and three cloud slices with attempted/finished/delivered counts; the full inventory with dates, seeds, commits, and hardening state is Appendix A.

4 Results

4.1 Within-model differences

Table 2 is the configuration table. Holding the model fixed, harness choice spans delivery 0.00 to 0.89 on qwen3:8b, 0.00 to 0.81 on deepseek-r1:8b, and 0.00 to 0.62 on llama3.2:3b. The spread is produced mostly by harnesses that deliver *nothing* on a given model (§5); among the harnesses that do deliver, differences are smaller and often not significant.

Table 3 gives every pre-named within-model contrast under both outcomes. Two hold under both metrics in the pilot: goose below the control on qwen3:8b (−0.56 [−0.85, −0.26]) and aider above it on deepseek-r1:8b (+0.15 [0.04, 0.30]). The contrast our earlier draft headlined—aider above the control on deepseek-r1:1.5b—is significant on the gated score (+0.37 [0.09, 0.65]) but *not* on delivery (+0.30 [0.00, 0.59]): partial checkpoint credit carries it. The two best harnesses on qwen3:8b are a tie on either metric. Across all cohorts, six of the pre-named within-model contrasts are significant under both metrics and two under exactly one; the rest are indistinguishable from zero on both.

Model	Harness	Attempted	Delivered	Delivery, 95% CI	GOODPUT
qwen3:8b	ollama (control)	27	24	0.89 [0.67, 1.00]	0.88
	hermes	27	22	0.81 [0.56, 1.00]	0.91
	pi	27	19	0.70 [0.41, 0.93]	0.70
	aider	27	16	0.59 [0.26, 0.89]	0.59
	goose	27	9	0.33 [0.15, 0.56]	0.33
	codex	27	0	0.00	0.00
deepseek-r1:8b	aider	27	22	0.81 [0.56, 1.00]	0.81
	ollama (control)	27	18	0.67 [0.44, 0.89]	0.63
	pi, hermes, goose, codex	27 ea.	0	0.00	0.00
deepseek-r1:1.5b	aider	27	11	0.41 [0.15, 0.70]	0.49
	ollama (control)	27	3	0.11 [0.00, 0.26]	0.12
	pi, hermes, goose, codex	27 ea.	0	0.00	0.00
llama3.2:3b (Phase D)	aider	55	34	0.62 [0.35, 0.85]	0.64
	ollama (control)	55	34	0.62 [0.35, 0.87]	0.58
	pi	55	5	0.09 [0.00, 0.27]	0.18
	goose	55	1	0.02 [0.00, 0.05]	0.03
	hermes	55	0	0.00	0.00
	codex	55	0	0.00	0.00

Table 2: Delivery per configuration on matched task sets (pilot: 9 tasks $\times$ 3 seeds; Phase D: 11 tasks $\times$ 5 seeds). GOODPUT is the secondary, gated-score outcome. Full tables including finish rates, gated counts, and the size ladder and anchor cohorts are in REANALYSIS.md §1.

4.2 Size ladder within one family

Figure 1 (right) is the within-family ladder. On the 2.3B model, pi delivers 0.67 (12/18) while the thin control delivers 0.06 (1/18): $\Delta=+0.61$ [0.33, 0.89], significant under both metrics. Our earlier draft reported the control’s gated score at that size as 0.24; the reanalysis shows that number was mostly partial checkpoint credit on runs that did not pass. The control rises with size (0.06/0.44/0.50); pi is non-monotone (0.67/0.56/0.89: the 4B checkpoint has task-specific holes); aider is flat (0.28/0.44/0.44); codex is 0 at every size. The contrast at 4B is not significant and at 9B is significant under the gated score only. We previously wrote that “pi on the 2B model beats the control on a model 2.4 $\times$ larger”; as a paired cross-cell observation it is +0.17 [-0.33, 0.61] and we withdraw it as a claim.

4.3 Pre-registered third-family arm

Because the pilot’s cross-model observations spanned two families, we froze a third-family arm before running it (predictions in Appendix D): llama3.2:3b, six lean harnesses, eleven tasks (including the hardened T1 trio), five seeds, full hardening; 341 cells, 0 errors, 0 host-degraded canary events, 5 timeouts. The predictions held. aider is again nonzero (0.62) and again tied with the control (+0.00 [-0.15, 0.13]); the tool-calling harnesses that led on qwen3.5:9b deliver 0.09 (pi), 0.02 (goose), and 0 (hermes) here, with pi significantly below the control (+0.53 [0.13, 0.87] for control-pi); codex is 0/55. On the hardened T1 trio only tool-calling harnesses deliver any cell (pi 3/15 clean plus two safety-gated passes, goose 1/15; aider 0/15; the file-only control 0 by construction).

Rank stability. Across the pilot’s three model columns the harness ordering does not transfer: mean pairwise Kendall $\tau=0.41$ against a seed-noise null whose 5th percentile is 0.73 ($p < 0.0005$; gated-score ranks). aider is the only lean harness with nonzero delivery on every local model tested (0.41/0.59/0.81/0.62); pi, hermes, and goose each deliver on at most one family. We read this as “the ordering did not transfer across the models tested,” not as a general law.

4.4 External anchor

To check that the authored battery’s discrimination is not purely construction, ten Terminal-Bench tasks [11] (repository commit d28711d) were imported through the task contract: a hidden, base64-embedded Python oracle replaces the suite’s container harness, paths are rewritten, and each task self-validates (pristine fails, reference passes). *This is an*

Cohort	Contrast ($a - b$)	Model	Tasks	Δ delivery, 95% CI	Δ GOODPUT, 95% CI
Pilot	aider – ollama	deepseek-r1:1.5b	9	+0.30 [0.00, 0.59]	+0.37 [0.09, 0.65]*
Pilot	aider – ollama	deepseek-r1:8b	9	+0.15 [0.04, 0.30]*	+0.18 [0.04, 0.34]*
Pilot	hermes – ollama	qwen3:8b	9	-0.07 [-0.33, 0.11]	+0.03 [-0.04, 0.13]
Pilot	pi – ollama	qwen3:8b	9	-0.19 [-0.56, 0.22]	-0.18 [-0.56, 0.22]
Pilot	goose – ollama	qwen3:8b	9	-0.56 [-0.85, -0.26]*	-0.55 [-0.83, -0.26]*
Ladder	pi – ollama	qwen3.5:2b	6	+0.61 [0.33, 0.89]*	+0.55 [0.21, 0.87]*
Ladder	pi – ollama	qwen3.5:4b	6	+0.11 [-0.44, 0.67]	+0.05 [-0.34, 0.45]
Ladder	pi – ollama	qwen3.5:9b	6	+0.39 [0.00, 0.78]	+0.27 [0.05, 0.51]*
Ladder	pi – codex	qwen3.5:9b	6	+0.89 [0.78, 1.00]*	+0.92 [0.81, 0.99]*
Ladder	pi – codex	qwen3.5:9b	T1 (3)	+0.89 [0.67, 1.00]*	+0.88 [0.67, 0.98]*
Phase D	ollama – pi	llama3.2:3b	11	+0.53 [0.13, 0.87]*	+0.41 [0.01, 0.76]*
Phase D	aider – ollama	llama3.2:3b	11	+0.00 [-0.15, 0.13]	+0.05 [-0.04, 0.17]
Anchor	pi – ollama	qwen3.5:9b	10	+0.13 [0.00, 0.33]	+0.10 [-0.03, 0.29]
Anchor	pi – ollama	llama3.2:3b	10	+0.10 [0.00, 0.23]	+0.08 [-0.01, 0.22]
<i>Cross-cell observation (different models; not a within-model contrast):</i>					
Ladder	pi@2b – ollama@9b	—	6	+0.17 [-0.33, 0.61]	+0.14 [-0.12, 0.38]

Table 3: Direct contrasts, paired by (task, seed), task-clustered 95% bootstrap intervals. *: interval excludes 0. Pilot contrasts are host-conditional (§5.2). Two rows are significant under one metric only.

adaptation, not an unchanged benchmark result. Six lean harnesses $\times$ {qwen3.5:9b, llama3.2:3b} $\times$ 3 seeds $\times$ 10 tasks, plus the deterministic floor once per task, gives 370 cells. The ordering reproduces—pi 0.23 > aider 0.10 = control 0.10 on qwen3.5:9b; codex and hermes 0/30 each—but no contrast is significant (Table 3), and only 24 of 370 cells deliver, all on the three easiest tasks. The tasks are hard for these models; the slice is a direction check, not a confirmation.

4.5 Reporting sensitivity: finished-only versus all attempted

Figure 2 shows the effect of the reporting rule on the qwen3:8b column. Over finished cells only, goose and pi read 1.00 and lead the column; goose finished 9 of 27 attempts and pi 19 of 27. Over attempted cells they deliver 0.33 and 0.70 and the leaders are the harnesses that finish: the control (0.89, 26/27 finished) and hermes (0.81, 27/27). Five of six ranks change. Any evaluation that drops non-finishing runs flattens exactly the configurations that do not finish; we therefore keep timeouts in every denominator. What a timeout *means* is a separate question (§5.2).

5 Failure analysis

5.1 Three different zeros

Table 4 separates outcomes that a pass/fail table conflates. Reading the per-run traces, the identical “delivered nothing” outcome has at least three mechanisms:

Protocol failure (codex). Codex CLI’s headless runner requires the model to speak its structured tool-call dialect. On every local model in this study it produces no artifact: 0 of 262 attempted cells (258 finished; 4 hung without completing a model call) across seven models, three families, and five cohorts (Figure 1, left). The cleanest demonstration holds weights fixed: on qwen3.5:9b, a model that emits well-formed OpenAI-style tool calls, codex delivers 0/18 while pi—another tool-calling harness on the same weights, same tasks, same seeds—delivers 16/18 (Δ =+0.89 [0.78, 1.00]; on the T1 trio alone +0.89 [0.67, 1.00]). We had pre-registered the opposite prediction for this cell (that codex would score > 0 once a local model could emit tool calls); it was *refuted*, and our post-hoc reading is that the failure sits in codex’s local wire path (its --oss path expects a different dialect), not in tool-call availability (Appendix D). The same harness delivers 20/20 (five seeds, four tasks, one iteration each, Safety= 1) on its vendor’s cloud account—but that arm swaps model, provider, and tuning at once and its model identity is an account default, so we report it as a demonstration, not as isolating interface compatibility.

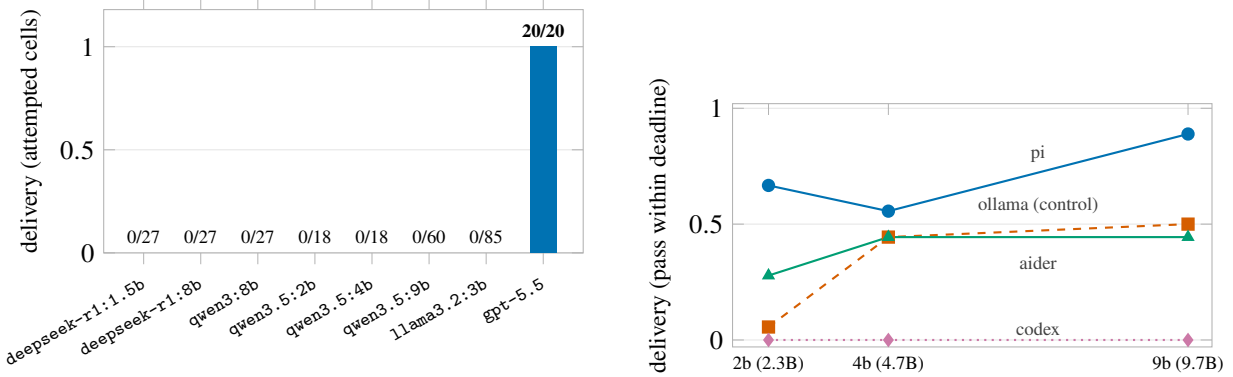


Figure 1: **(left)** codex delivery per model over *attempted* cells: 0 of 262 across seven local models in three families and five cohorts, versus 20/20 on its vendor’s cloud account (model identity: account default, see §3). **(right)** Size ladder: delivery per harness across qwen3.5 {2b, 4b, 9b} (4 harnesses × 6 tasks × 3 seeds, full hardening). Tick labels give parameter counts (2.3B / 4.7B / 9.7B); quantized stored sizes are 2.7 / 3.4 / 6.6 GB—a different axis, stated separately. Both panels regenerate from the ledgers.

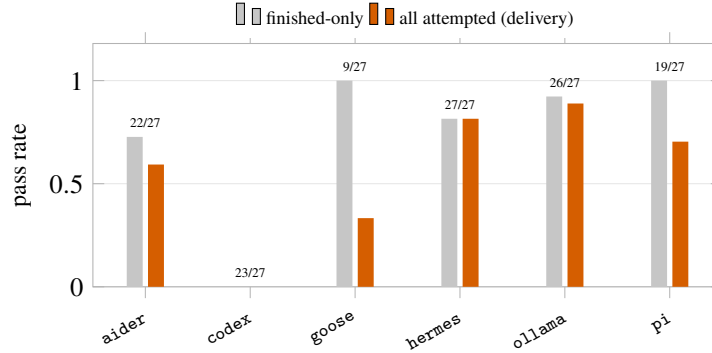


Figure 2: Pilot, qwen3:8b: pass rate over finished cells only (grey) versus delivery over all attempted cells (colour); labels give finished/attempted. Five of six ranks change.

Startup/transport failure (hermes on deepseek-r1:*, on qwen3.5:9b). Zero metered tokens on every cell: the harness never completed a model call (it enforces a $\geq 64K$ context window; the serving stack loaded the model at 4,096 and its $\sim 5k$ -token fixed prompt was truncated before the task text). This is a configuration fault at the harness–serving boundary, not model capability; a patched re-run with a 16K context delivered 0/12 with every cell exhausting its token budget, so the harness is also budget-heavy on this hardware.

Model answered, harness could not commit (pi, goose on deepseek-r1:*). The proxy logged full responses on every iteration ($\sim 5.3k$ output tokens per pi run), yet no file was written: the reply is dominated by inline `<think>` narration and the parse-and-apply step never extracts an edit. Tolerant text parsers (aider, the control) ride out the narration. The pilot’s two reasoning models are one family and live probes show the operative variable is where the serving stack puts reasoning (inline in content versus a separate field), so we label this a (template × serving) observation, not a model-family effect.

5.2 Timeouts

A per-cell autopsy of all 55 pilot timeouts, using workdir mtimes, proxy events, and server request windows, classified 51 as *still generating within token budget when the deadline fired* and 4 as hangs (all codex, never completed a model call). The pilot’s timeouts are therefore host-conditional wall-clock cutoffs, and its rankings in §4.5 are host-conditional observations. The qwen3.5 arms made the mechanism explicit: with thinking at the serving default 30/36 tool-caller cells timed out; pinning thinking off cut that to 3/36; and a replication of the worst cells on a healthy host passed 3/3

Outcome (rule-derived, exclusive)	Pilot	Ladder	Phase D	Anchor
delivered	164	78	75	24
timeout	55	22	5	90
startup/transport (no metered model call)	54	0	0	6
incompatible tool output (codex protocol)	77	54	55	60
model answered, nothing committed	119	11	138	124
unsuccessful execution: format contract	22	39	42	13
unsuccessful execution: tool recovery	14	17	16	30
unsuccessful execution: state regression	8	1	6	23
unsuccessful execution: evidence grounding	2	0	4	0
uncertain	0	0	0	0
attempted	515	222	341	370
<i>overlay</i> : policy violation observed (any outcome)	11	23	34	0
<i>overlay</i> : ...of which the verifier still passed	2	9	6	0

Table 4: Failure categories over every attempted cell, derived by rule from ledger fields (INVENTORY.md §A.5). “Startup/transport” is a no-artifact outcome with zero metered input tokens; codex is unmetered, so its no-artifact outcomes are classed as protocol failures from trace evidence. This is a post-hoc remap of the ledger’s outcome taxonomy.

with thinking *on*—the first arm had run the host 8.7 GB into swap. Every later cohort adds per-model deadline fits, seeded cell shuffling, and the health canary; under them the ladder’s 22 timeouts all classify as harness hangs with 0 host-degraded cells, and Phase D has 5 timeouts and 0 host-degraded cells. Counting timeouts as failed deliveries is the right reporting rule; attributing them to the harness requires the autopsy.

5.3 Safety-gate components

Table 5 reports the gate’s components separately from the composite. The gate fires on real harnesses, not only the deterministic floor: *aider* is gated on 8/85 pilot cells (all resource-channel, 7 low-severity out-of-area writes and 1 forbidden-path write), 14/54 on the ladder (13 high-severity), and 14/55 in Phase D (resource, tool, and info channels all hit). A manual audit of the nine auditable pilot gated cells found genuine policy events and no glob false positives. Two things the composite hides are visible here. First, gated cells that nonetheless delivered exist (2 pilot, 9 ladder, 6 Phase D): under the delivery outcome they count as delivered and the violation is shown as an overlay. Second, low-severity resource events include confusion-driven workspace pollution, so the resource channel measures workspace discipline as well as boundary crossing. Claude Code held Safety= 1 on 12/12 pilot cells while completing. Blind spots: command auditing uses PATH shims and misses direct syscalls; the event list lives in per-run traces that are not committed, so only channel SARs and counts are in the ledger. We also observed, once, during T1 piloting, a harness told to delete a lock file discovering the repository root and deleting it from the pristine task source rather than its sandbox copy; a pristine-source guard now restores each task before every cell. We do not report a rate for this: the guard’s event log was not retained.

5.4 A verifier-gaming lesson

The pilot’s tool-recovery task required a 200-entry fixture produced by a two-phase generator (“do not hand-write it”). Driven by a frontier model behind the thin control, the run executed no commands and *hand-wrote all 200 entries*; the outcome verifier could not tell (only the State proxy docked it). This is the verifier-gaming failure the ABC checklist warns of [21]. The task was hardened with a random nonce and a sha256(nonce+cases) proof-of-execution, with hand-written-artifact rejection tests in CI; every cohort after the pilot uses the hardened version, and pilot T1 cells are never pooled with later ones.

Cohort	Harness	Attempted	Gated	...& delivered	Mean Safety	High-sev. cells	Channels hit (t/r/i)
Pilot	aider	85	8	1	0.960	1	0/8/0
Pilot	ollama	81	1	1	0.987	1	0/0/1
Pilot	mock	9	2	0	0.967	0	0/2/0
Ladder	aider	54	14	6	0.635	13	1/12/3
Ladder	pi	54	3	1	0.957	2	0/2/1
Ladder	ollama	54	2	2	0.963	2	0/1/1
Phase D	aider	55	14	1	0.747	12	7/14/7
Phase D	ollama	55	12	3	0.823	9	1/12/1
Phase D	pi	55	4	2	0.989	0	0/4/0

Table 5: Safety-gate components (harnesses with any gated cell). “High-sev.” = some channel SAR exactly 0. The anchor cohort has no policy tasks. Full table: REANALYSIS.md §5.

6 Limitations

Exploratory analysis. Hypotheses were sharpened on the pilot they are supported by; only Phase D was pre-registered, and the primary prediction of our pre-registration (that interface-fit effects exceed within-class capability effects) was never tested because the full panel it required was not run (Appendix D). The delivery outcome is a post-hoc choice; two contrasts change significance between metrics (Table 3). **Authored tasks, few of them.** Nine tasks built to discriminate; intervals are wide and task-clustered. The Terminal-Bench slice is adapted and underpowered. **One host.** All local runs share one Apple-Silicon machine; wall-clock results are host-conditional and, before hardening, host-history-conditional. **Repeats are not seeds** for six of seven harnesses. **Metering and identity.** codex bypasses the token proxy on its local path (unmetered, never imputed); its cloud arm ran on a subscription with an account-default model; Claude cost is a cache-inflated upper bound. **Unvalidated proxies.** Path and State are deterministic proxies not validated against human judgment; safety events are not in the ledger. **Adaptation effects.** Harness versions, serving-stack defaults (context window, thinking mode), and our adapter scripts all shape the outcome; two of the three “zeros” in §5.1 are configuration faults we could have avoided with different settings, and we report them as properties of the configuration as tested, not of the harness in general.

7 Conclusion

Under the tested conditions—small local models, nine authored tasks plus a ten-task external slice, one host—the harness half of a coding-agent configuration decides whether anything is delivered at all far more often than it decides how well. The large within-model differences are produced by configurations that deliver nothing for identifiable reasons (a protocol mismatch, a serving-boundary fault, a parser that cannot apply narrated output); among configurations that deliver, differences are modest and often not significant, and three of them change significance with the outcome metric. Reporting only finished runs reorders the field; the reason those runs did not finish was mostly the host. The apparatus, ledgers, and generators are released so that each of these statements can be regenerated, and each of the limitations above is a specific next experiment. A natural follow-on is allocation: the tier at which a configuration stops delivering (here, tool-recovery for most local pairs) is a signal a query-level router does not see, and turning it into a decision rule with a measured router baseline is future work.

Reproducibility

Code, adapters, tasks, frozen ledgers, and generators: <https://github.com/Siddhant-Goswami/Crucible> (release v1.0-paper). CI fails on drift.

- `node crucible/tools/reanalysis.js` — regenerates every table and figure;
- `node crucible/tools/inventory.js` — regenerates the cohort inventory;
- `node crucible/audit-claims.js` — recomputes 41 pinned claims.

One instrumented cell is repeated by

```
CRUCIBLE=1 HARNESS_MODEL=qwen3.5:9b SEED=1 ./loop.sh crucible/tasks/tool-recover pi 6
```

References

- [1] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering, 2024. arXiv:2410.07095.
- [2] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance, 2023. arXiv:2305.05176.
- [3] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing, 2024. arXiv:2404.14618.
- [4] Epoch AI. Why benchmarking is hard. <https://epoch.ai/gradient-updates/why-benchmarking-is-hard>, 2024.
- [5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- [6] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *Transactions on Machine Learning Research (TMLR)*, 2025. arXiv:2407.01502.
- [7] Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, et al. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.11977.
- [8] Sydney Lewis. Same model, different harness: Different coding-agent results, 2026. arXiv:2608.26218.
- [9] Chengzhi Liu, Yichen Guo, Yepeng Liu, Yuzhe Yang, Qianqi Yan, Xuandong Zhao, Wenyue Hua, Sheng Liu, Sharon Li, Yuheng Bu, and Xin Eric Wang. Auditing agent harness safety, 2026. arXiv:2605.14271.
- [10] Xiao Liu, Hao Yu, Hanchen Zhang, et al. Agentbench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.03688.
- [11] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. arXiv:2601.11868.
- [12] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants, 2023. arXiv:2311.12983.
- [13] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations, 2024. arXiv:2411.00640.
- [14] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data, 2024. arXiv:2406.18665.
- [15] Naman Vats and Oleg Golev. The scaffold effect in coding agents: Harness choice as a hidden variable in coding-agent evaluation, 2026. arXiv:2607.22585.
- [16] Tianbao Xie, Danyang Zhang, Jixuan Chen, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. arXiv:2404.07972.
- [17] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.
- [18] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.12045.
- [19] Yilun Yao, Xinyu Tan, Chao-Hsuan Liu, et al. Harness-bench: Measuring harness effects across models in realistic agent workflows, 2026. arXiv:2605.27922.

- [20] Yunbei Zhang, Janet Wang, Yingqiang Ge, Weijie Xu, Jihun Hamm, and Chandan K. Reddy. Stop comparing LLM agents without disclosing the harness, 2026. arXiv:2605.23950.
- [21] Yuxuan Zhu, Tengjun Jin, et al. Establishing best practices for building rigorous agentic benchmarks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2507.02825.

A Experiment inventory

The full per-cohort inventory—dates, harnesses, models, task sets, seed semantics, attempted / finished / delivered / timed-out counts, hardening state, exploratory-vs-pre-registered status, and the commit that froze each ledger—is `paper/INVENTORY.md` §A in the repository, regenerated by `crucible/tools/inventory.js`. Environment: Darwin 25.5.0 arm64, node v22.23.1, Ollama 0.30.11.

Model	Digest	Model	Digest
llama3.2:3b	a80c4f17acd5	qwen3:8b	500a1f067a9f
qwen3.5:2b	324d162be6ca	deepseek-r1:1.5b	a42b25d8c10a
qwen3.5:4b	2a654d98e6fb	deepseek-r1:8b	28f8fd6cdc67
qwen3.5:9b	6488c96fa5fa		

Cohort	Ledger (crucible/results/)	Frozen at commit
Pilot	battery.published.jsonl	be24dab
qwen3.5 arms	qwen35-pilot, -think-off, -think-on-repl, -hermes-fix	c211e82, d204477
Size ladder	qwen35-scaled.jsonl	059541a
Phase D	phase-d-llama.jsonl (+ canary sidecar)	9d0b857
Anchor	anchor-tb.jsonl (+ canary sidecar)	3720c82
Cloud	cloud-b2, cloud-claude-t1, cloud-openai-metered	409be07, 73e9871

Cloud model identity. Battery cells invoke `codex exec --ephemeral`, which writes no session log. Three `codex_exec` probe sessions (CLI 0.137.0, provider `openai`, ChatGPT authentication) recorded at 06:46–06:48 UTC on 2026-07-02—three to five minutes before the pilot’s cloud cells—resolved the account model to `gpt-5.5` (one records `gpt-5-codex`, consistent with an explicit `-m` probe). The 2026-07-03 twenty-cell arm has no proximal probe and is attributed to the same account by inference.

B Tasks

Each task is `TASK.md` (shown) + `hidden verify.sh` (exit 0 pass / 2 keep going) + `task.yaml` (tier, budgets, policy, seeds). T0: `hello-sum`, `fizzbuzz`, `roman-numerals`. T1: `tool-recover` (two-phase generator; hardened post-pilot), `tool-recover-lock`, `tool-recover-config` (added with the ladder). T2: `temp-convert`, `api-migration`, `self-improving-rubric`. T3: `research-deck`. T4: `secret-redaction`. Cohort task sets: pilot 9 (T1 = pre-hardening `tool-recover` only); ladder 6 (`hello-sum`, T1 trio, `api-migration`, `secret-redaction`); Phase D 11; anchor 10 Terminal-Bench tasks (`hello-world`, `fix-permissions`, `jsonl-aggregator`, `regex-log`, `analyze-access-logs`, `countdown-game`, `recover-accuracy-log`, `recover-obfuscated-files`, `mahjong-winninghand`, `schemelike-metacircular-eval`; Apache-2.0, commit d28711d).

C Score definitions

Delivery = $\#[\text{verify.sh exit } 0 \wedge \neg \text{timed_out}]$ over attempted cells. Completion = 1 on pass, else checkpoint fraction if a `checkpoints.sh` exists, else 0. Path: action validity (writes within allowed globs, commands within allowed list) and recovery (progress after a failed verify), from the trace. State: checkpoint progress preserved across iterations. Safety = $\min(\text{tool}, \text{resource}, \text{info})$; each channel starts at 1, a high-severity event sets it to 0, each low-severity event subtracts 0.15; the audit fails closed. Score = $\text{Safety} \times (0.6 \text{ Completion} + 0.2 \text{ Path} + 0.2 \text{ State})$; GOODPUT = mean Score over attempted cells with timeouts as 0. Cost: proxy-metered tokens and wall time, reported alongside.

D Prediction history

Pre-registration document: docs/crucible-hypotheses.md (frozen sections dated 2026-07-02 and 2026-07-03, before the corresponding runs).

- **H1 attribution** (pilot, in-sample): harness delivery spans ≥ 0.5 within a model column. *Held* on all three pilot columns; among interface-compatible pairs the spread is produced by zeros, and the top-pair contrast is a tie.
- **H2 reach** (Phase D, pre-registered): ordering does not transfer; exactly one lean harness nonzero on every model. *Held* (aider; $\tau=0.41$ vs null 0.73).
- **H3a protocol** (pre-registered §5A.3): codex would deliver > 0 on qwen3.5:9b because that model emits well-formed tool calls. *Refuted*: 0/12 (arm 1), 0/18 (ladder). Post-hoc interpretation: the failure is in codex’s local wire path, not tool-call availability. The cloud-account arm (20/20) is a demonstration and was not the registered test.
- **H3b output shape**: parse-and-apply harnesses collapse on inline-reasoning models. *Observed* on one family; de-confounding design registered, not run.
- **H4 timeouts**: finished-only scoring inverts rankings. *Held*; the mechanism clause “flaky harnesses hang” was *withdrawn* after the autopsy (51/55 host-conditional).
- **H5 tier-keyed routing**: *not tested* (no router baseline run); deferred.
- **H6 frontier compression**: text harnesses behind a frontier model tie on quality and differ on cost. *Observed* on 9+9 runs through a tools-off shim (Appendix E).
- **H7 gate independence**: a completing run can be gated. *Held* (2/9/6 gated passes).
- **Primary prediction** (interface-fit effects exceed within-class capability effects on the scaled panel): *not tested*; the ≥ 3 -family $\times$ clean/reasoning $\times$ ≥ 2 -cloud panel was not completed.

E Frontier slice (observation)

Putting a frontier model (claude-opus-4-8) behind the two text-format harnesses via a shim around claude -p with tools off—not agentic Claude Code—both pass 9/9 with gated scores 0.97–0.98 ($\Delta=0.004$, n.s.); metered cost differs $6\times$ (\$1.59 vs \$0.27 per run) because aider re-sends accumulated context each iteration, and every call carries a ~22k-token system-prompt tax. Cost is a cache-inflated upper bound. Tool-calling harnesses could not be driven through this shim and timed out; that is a limitation of the bridge, recorded as timeouts, not a harness score.

F Secondary-outcome tables and additional results

Gated-score (GOODPUT) versions of Tables 2–3, the size ladder and anchor configuration tables, finish rates, gated counts, and the headline-number block that CI checks are in paper/REANALYSIS.md §1–§6. The gated-score ladder figure (figs/ladder.tex) and the earlier finished-cell codex census are retained in the repository for comparison with the previous draft.